

PSP Actividad DUAL

Desarrollo de Servicio REST Seguro

Módulo: Programación de Servicios y Procesos

Ciclo: 2º DAM

Curso: 2025/2026

Índice

Actividad DUAL - Programación de Servicios y Procesos (PSP)

1. Desarrollo de un servicio REST sencillo (RA4)

2. Añadir medidas básicas de seguridad al servicio REST (RA5)

Actividad DUAL - Programación de Servicios y Procesos (PSP)

Proyecto: API REST de Gestión de Productos (`api-productos`)

1. Desarrollo de un servicio REST sencillo (RA4)

1. Elección del Framework:

Para el desarrollo de la API REST se ha optado por el framework **Spring Boot** utilizando Java 21, al ser el estándar principal empleado en el entorno profesional para la creación de microservicios robustos.

2. Entidad Gestionada:

La aplicación desarrolla una API REST orientada a la gestión y mantenimiento de un catálogo de **Productos**. Las propiedades fundamentales de cada producto incluyen: `id`, `nombre`, `descripcion`, `precio` y `stock` (cantidad disponible).

3. Implementación de Operaciones Básicas (CRUD):

Se ha implementado un servicio RESTful completo en la clase `ProductoController.java`, ofreciendo los siguientes *endpoints* para interactuar con el recurso:

- `GET /api/productos` : Retorna la lista de todos los productos del catálogo.
- `GET /api/productos/{id}` : Obtiene la información detallada de un producto específico.
- `POST /api/productos` : Crea y añade un nuevo producto a la base de datos.
- `PUT /api/productos/{id}` : Modifica y actualiza la información de un producto preexistente.
- `DELETE /api/productos/{id}` : Elimina permanentemente un producto del catálogo.

4. Pruebas y Funcionamiento Funcional:

El funcionamiento del servicio se verificó íntegramente mediante la herramienta **Postman**. Todas las rutas y sus *payloads* JSON se han documentado en una colección `postman_collection.json` exportada junto al proyecto, garantizando respuestas HTTP estándar (`200 OK` en aciertos, `404 Not Found` o `400 Bad Request` en fallos).

2. Añadir medidas básicas de seguridad al servicio REST (RA5)

1. Sistema de Autenticación:

Se ha dotado a la API de un sistema de seguridad implementando HTTP Basic Authentication a través de **Spring Security**. El acceso se gestiona mediante la clase `SecurityConfig.java`, donde se ha creado en memoria (para fines de esta práctica) un usuario con privilegios de administrador:

- **Usuario:** `admin`
- **Contraseña:** `admin123` (encriptada con BCrypt)

2. Restricción de Acceso y Rutas Protegidas:

Se ha implementado un esquema de autorización y protección de acceso basado en roles aplicados a las rutas HTTP de la API, cumpliendo con el principio de mínimo privilegio:

- **Rutas Públicas:** Se ha configurado el acceso sin restricciones (público) a las operaciones de consulta pasiva:
 - `GET /api/productos` (Cualquier cliente puede ver el catálogo)
 - `GET /api/productos/{id}`
- **Rutas Protegidas:** El sistema exige estar autenticado como `ADMIN` para operaciones que alteran el estado o la información (inserción, actualización, eliminación):
 - `POST`, `PUT` y `DELETE` hacia `/api/productos/**` serán interceptados y denegarán el acceso inmediato (`HTTP 401 Unauthorized`) si el usuario no aporta las credenciales correctas en la cabecera HTTP.

3. Validación de Datos (Data Validation):

Para garantizar la integridad y coherencia de la información almacenada en el sistema, se han aplicado reglas de **validación estricta** a las peticiones entrantes utilizando las anotaciones de validación nativas (`jakarta.validation`). El objeto de transferencia `ProductoDTO` incluye las siguientes restricciones de validación:

- `@NotBlank` : El `nombre` del producto no puede estar vacío.
- `@NotNull` y `@Positive` : El `precio` debe existir obligatoriamente y no puede ser negativo ni 0.
- `@Min(0)` : El `stock` no puede tomar valores por debajo de 0 (no concebimos inventario negativo).

Si una solicitud `POST` o `PUT` incumple estas reglas, Spring rechaza automáticamente la inyección de datos retornando un error HTTP `400 Bad Request` .

4. Documentación de Seguridad (Evidencias de Funcionamiento):

A continuación, se adjuntan las capturas correspondientes usando el cliente a terminal `ClienteConsola.java` y/o Postman:

- **Acceso Permitido (200 OK):** *(Añade aquí tu captura en Word demostrando una petición POST o GET exitosa autenticándote con admin:admin123)*
- **Acceso Denegado (401 Unauthorized):** *(Añade aquí tu captura en Word demostrando el intento de crear o borrar un producto sin estar logueado o introduciendo datos inválidos que devuelven un 400 Bad Request)*

Conclusión de la Actividad:

El desarrollo consolida los resultados de aprendizaje (RA4 y RA5) creando un servicio distribuido eficiente en red, que a la par se encuentra asegurado criptográficamente y protegido frente a alteraciones no validadas o usuarios sin permisos adecuados.